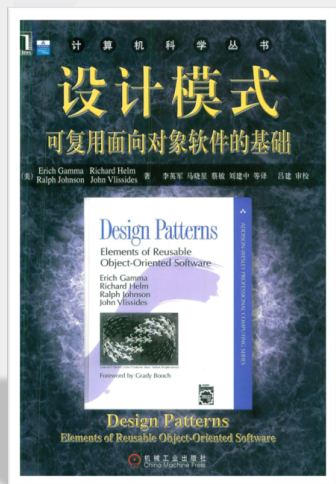




设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

Decorator（装饰模式）

1 Intent（意图）

动态地给对象附加额外职责。

Decorator 提供了一种 **比继承更灵活的功能扩展方式**。

通过在对象外部包裹新的对象，可以在不改变原对象接口的情况下扩展其行为。

在嵌入式系统中，可以理解为：

在核心模块外部逐层附加功能模块，使行为能够按需叠加，而无需修改原始模块。

2 Also Known As（别名）

Wrapper（包装器）

3 Motivation（动机）

在系统设计中，经常需要为某个模块增加附加功能，例如：

- 增加日志
- 增加校验
- 增加加密
- 增加统计
- 增加缓存

如果通过继承扩展功能，可能会产生大量组合类。例如：

- UARTWithCRC

- UARTWithLog
- UARTWithCRCAndLog
- UARTWithEncryptAndCRC

当功能组合增多时，类的数量会迅速膨胀。

Decorator 通过 **对象组合** 的方式解决这个问题。

每个增强功能独立实现，通过包装对象的方式逐层叠加。

例如：

```
Log
↓
Encrypt
↓
CRC
↓
UART
```

这样系统可以灵活组合不同功能，而无需创建大量派生类。

4 Applicability（适用性）

Decorator 适用于以下情况：

1 需要在运行时动态增加对象职责

功能可能按不同组合出现，而不希望在编译期固定。

2 希望避免大量子类

当功能组合较多时，继承会导致类型数量爆炸。

3 需要在不修改原对象的情况下扩展功能

Decorator 可以在保持接口不变的情况下增强行为。

4 多个增强功能可以独立实现

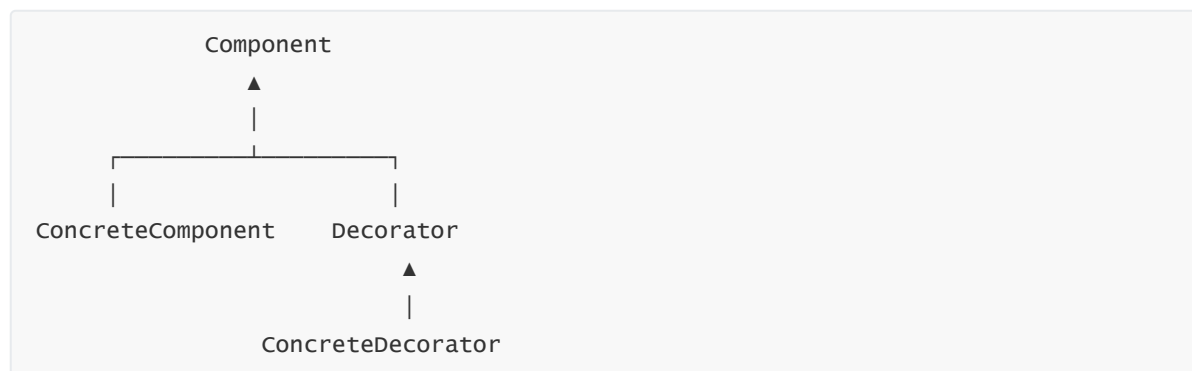
每个功能模块可以单独开发，并按需组合。

在嵌入式系统中，典型场景包括：

- 通信发送增强（CRC / 加密 / 日志）
 - 存储访问增强（缓存 / ECC / 加密）
 - 传感器数据处理（滤波 / 校准 / 平滑）
 - 日志输出增强（时间戳 / 颜色 / 异步缓冲）
-

5 Structure (结构)

Decorator 模式包含以下基本结构：



Decorator 与 Component 具有相同接口，同时内部持有一个 Component 对象。

因此系统可以形成 **递归包装结构**。

6 Participants (参与者)

Component

定义对象接口。

客户端通过该接口访问对象。

ConcreteComponent

实现基础功能的对象。

它是被装饰的核心对象。

Decorator

实现 Component 接口，并持有一个 Component 对象。

请求会被转发给该对象。

ConcreteDecorator

在转发请求时附加新的职责。

不同装饰器提供不同的增强行为。

7 Collaborations (协作)

客户端通过 Component 接口访问对象。

一个 ConcreteDecorator 接收请求后：

1. 执行自身增强逻辑
2. 将请求转发给内部的 Component

3. Component 继续处理请求

多个 Decorator 可以嵌套形成调用链。

例如：

```
Client
↓
Decorator A
↓
Decorator B
↓
ConcreteComponent
```

如果存在前后处理逻辑，则调用过程可能为：

```
A before
B before
Core operation
B after
A after
```

8 Consequences (效果)

Decorator 模式带来以下影响。

优点

1 比继承更灵活

功能可以通过组合动态叠加，而不需要创建新的子类。

2 支持功能组合

不同增强功能可以按需组合。

3 遵循开闭原则

系统可以在不修改原对象的情况下增加新功能。

4 职责划分清晰

每个 Decorator 只负责一个增强行为。

缺点

1 对象结构更复杂

系统可能形成较深的装饰链。

2 调试难度增加

行为分布在多个装饰层中。

3 行为顺序敏感

某些装饰器的执行顺序可能影响结果。

4 可能增加运行开销

在嵌入式系统中，多层调用可能增加函数调用成本。

9 Implementation（实现）

Decorator 的实现通常包含以下关键点：

1 统一接口

所有组件必须遵循同一接口。

2 装饰对象持有同接口对象

Decorator 内部必须保存一个 Component 引用。

3 请求转发

Decorator 在执行自身逻辑后，将请求转发给内部对象。

4 可递归组合

Decorator 本身也是 Component，因此可以继续被包装。

在嵌入式 C 中，这种结构通常通过：

- 接口结构体
- 函数指针表
- 对象组合

来实现。